

OQA Contracts Audit Report



Updated: June 17, 2026

Table of Contents

- [Introduction](#)
 - [Disclaimer](#)
 - [Scope of Audit](#)
 - [Methodology](#)
 - [Security Review Summary](#)
 - [Areas of Focus](#)
 - [OQAFixedDeposits.sol](#)
 - [OQAFixedDepositsPayout.sol](#)
 - [OQAAgentPayoutVault.sol](#)
 - [Security Analysis](#)
 - [OQAFixedDeposits.sol](#)
 - [OQAFixedDepositsPayout.sol](#)
 - [OQAAgentPayoutVault.sol](#)
 - [Contract Structure](#)
 - [OQAFixedDeposits.sol](#)
 - [OQAFixedDepositsPayout.sol](#)
 - [OQAAgentPayoutVault.sol](#)
-

Introduction

The purpose of this report is to document the findings from the security audit of the OQA (Offshore Quant Account) fixed deposit contract suite. OQA is a standalone variant of the OBA(Offshore Builders Account) product line that does not integrate with City of Atlantus contracts — there is no on-chain user-ID resolution, EVT tier lookup, or genesis-key NFT reward system. Identity, whitelisting, APY assignment, and per-user deposit limits are all tracked directly against wallet addresses and are set via backend-signed EIP-712 messages rather than resolved from ecosystem NFT holdings. This audit examines deposit lifecycle management, the quarter-gated yield accrual model, partial and full withdrawal accounting, payout request flows, and the independent agent commission vault.

Disclaimer

This report is based on the information provided at the time of the audit and does not guarantee the absence of future vulnerabilities. Subsequent security reviews and on-chain monitoring are strongly recommended.

Scope of Audit

The audit focused on the following contracts:

- [OQAFixedDeposits.sol](#)
- [OQAFixedDepositsPayout.sol](#)
- [OQAAgentPayoutVault.sol](#)

The following aspects were reviewed:

- Security mechanisms, including EIP-712 signature validation, nonce management, and role-based access control across all three contracts
 - Code correctness, fund accounting integrity, and logical flow for full and partial deposit withdrawals
 - Quarter-gated pending-yield unlock logic and its interaction with early withdrawal and APY rate changes
 - Adherence to upgradeable contract best practices (EIP-7201 namespaced storage, storage gaps)
 - Split-claim integrity and replay protection in the agent commission vault
 - Gas efficiency and error handling
-

Methodology

The audit process involved:

- Manual code review of inheritance, signature hashing, yield-checkpoint logic, and request lifecycle flows
 - Automated analysis using Slither and Aderyn to detect common vulnerabilities such as reentrancy, access control gaps, and unsafe arithmetic
 - Scenario-based testing using Foundry for full/partial withdrawals, quarter-boundary yield unlocking, APY rate changes mid-cycle, compounding, and agent split-claim edge cases
-

Security Review Summary

Review commit hash:

- `5c79e521b7b42efdaad6f395ce8704896b2e8b73` **Contracts in Scope:**
- `OQAFixedDeposits.sol`
- `OQAFixedDepositsPayout.sol`
- `OQAAgentPayoutVault.sol`

The following number of issues were found, categorised by their severity:

- **High: 0 issues**
 - **Medium: 0 issues**
 - **Low: 0 issues**
-

Areas of Focus

Given that OQA removes the COAAuth trust layer and introduces a materially different yield-accrual model from the rest of the ecosystem, review effort was deliberately weighted toward the areas most likely to hide subtle accounting or authorization bugs rather than spread evenly across the codebase.

OQAFixedDeposits.sol

The bulk of review time went into the quarter-gated pending-yield mechanism (`_checkpointPendingYield`, `_unlockPendingYield`, `_latestUnlockedYieldCheckpoint`, `_nextYieldUnlockTime`). This is a

fundamentally different accrual model from the continuous per-second yield used elsewhere in the ecosystem, and boundary conditions around quarter transitions were tested extensively — in particular, what happens when a checkpoint, an APY rate change, and a quarter-unlock boundary all land on the same block, and whether yield could be double-counted or silently dropped when `pendingYield` is unlocked into `yield` while a fresh `_calcYieldSince` calculation runs in the same call.

Equal attention was paid to the new partial-withdrawal path. Because `_consumeWithdraw` now supports withdrawing less than the full principal, the interaction between `_enforceRemainingDepositAmount` (which blocks a remaining balance from dropping below the user's effective minimum) and the maturity-based branching (full yield checkpoint on matured deposits vs. `_clearYieldAfterEarlyWithdraw` + `yieldStopped` on early withdrawals) was traced carefully to confirm a user cannot partially withdraw early, get yield zeroed out, and then continue accruing on the remaining principal in a way that misrepresents their actual yield history.

Signature-gated administrative paths (`addToWhitelist`, `removeFromWhitelist`, `setUserApy`, `depositOnBehalf`) were each checked for correct nonce consumption ordering relative to the EIP-712 digest construction, since nonces here are taken from `msg.sender` (the relayer) rather than the subject `userAddress` — a deliberate design choice that was verified against replay scenarios involving multiple relayers acting on behalf of the same user.

OQAFixedDepositsPayout.sol

Review here concentrated on the dual `consumeWithdraw` overload surface and confirming that `_requestWithdraw`'s `withdrawAll` branching always routes to the correct `OQAFixedDeposits` function selector — a mismatch here would either silently cap a "withdraw all" request to zero or allow an unintended partial amount to bypass full-withdrawal accounting. The single-asset `_distribute` split logic was checked against rounding-dust scenarios where basis-point splits don't divide evenly, to confirm no value is permanently stranded in the contract.

OQAAgentPayoutVault.sol

Focus centred on the split-claim flow (`claimCommissionWithSplit`), specifically whether the `splitsHash` binding inside the EIP-712 digest fully constrains the `receivers/amounts` arrays such that a relayer cannot reorder, substitute, or pad either array post-signature. The `ContractReceiversNotAllowed` check was also stress-tested against proxy and minimal-proxy receiver patterns to confirm `code.length` checks at claim time cannot be bypassed by a receiver that is a contract during signing but not yet deployed at claim time, or vice versa.

Security Analysis

OQAFixedDeposits

`OQAFixedDeposits` is the standalone accounting core for the OQA product line. Unlike its COA-integrated counterpart, it has no external identity contract dependency — every user-facing mapping (`whitelistStates`, `userApy`, `userMinDepositAmounts`, `userMaxDepositAmounts`, `userDepositIds`) is keyed directly by wallet address. All state lives inside a single EIP-7201 namespaced `OQAFixedDepositsStorage` struct, preserving upgrade safety without relying on a separate manager or getters contract.

Because there is no on-chain EVT/NFT resolution, every privileged state change — whitelisting, APY assignment, and admin-initiated deposits — is authorized through backend-signed EIP-712 messages verified via the `OQASigVerifier` library, with replay protection enforced through `nonces[msg.sender].setUserApy` correctly checkpoints yield on every active deposit for the affected user (`_checkpointYieldForRateChange`) before applying the new rate, ensuring no yield is lost or double-applied across an APY change.

The yield model introduces a pending-yield concept not present in the OBA contracts: yield accrued within the current, not-yet-completed quarter is held in `pendingYield` and only folded into the confirmed `yield` balance once the corresponding quarter boundary (`pendingYieldUnlockTime`) has passed. This is consistently applied across `_updateYield`, `_checkpointPendingYield`, and the view-side `_getAccumulatedYield`, so on-chain claimable balances and off-chain projections remain consistent. Early withdrawal correctly halts further accrual via `yieldStopped` and zeroes pending/claimed/confirmed yield through `_clearYieldAfterEarlyWithdraw`, while matured withdrawals preserve and checkpoint yield normally.

Deposit limits are enforced per-user (`userMinDepositAmounts` / `userMaxDepositAmounts`) layered on top of a protocol-wide `minimumDepositAmount` floor, with `_enforceDepositAmount` and `_enforceRemainingDepositAmount` consistently applied across initial deposits, top-ups, and partial withdrawals — preventing a partial withdrawal from leaving a dust balance below the user's configured floor.

Automated tools (Slither/Aderyn) flagged no high-severity issues. Testing confirmed correct behaviour across full withdrawal, partial withdrawal, quarter-boundary yield unlocking, mid-cycle APY changes, and compounding flows.

OQAFixedDepositsPayout

`OQAFixedDepositsPayout` mirrors the two-phase async request pattern used across the ecosystem: a user-initiated request, a backend-signed fulfilment (`processWithdrawRequests`), and a final user-initiated completion that moves funds. This separation ensures no token transfer occurs without an off-chain-validated, on-chain-verified signature in between.

The contract supports both full and partial principal withdrawal through two `requestWithdraw` overloads, correctly routing to the matching `consumeWithdraw` signature on `OQAFixedDeposits`. Because OQA has no gPRLZ/cashier dual-asset model, payout distribution is simplified to a single-asset split (`_distribute`), with basis-point allocations validated to sum exactly to `BP` before any transfer executes. `compoundYield` correctly pulls any wallet top-up directly to the custodian rather than routing it through the payout contract's own balance, keeping payout liquidity isolated from new-deposit principal.

All admin fund movements (`adminDepositToken`, `withdrawToken`) remain signature-gated identically to the COA-integrated payout contract, meaning `OPERATOR_ROLE` alone is insufficient to move funds without a valid signer-produced EIP-712 signature.

OQAAgentPayoutVault

`OQAAgentPayoutVault` is a self-contained commission vault, independent of the fixed-deposit accounting contracts, used to pay agents their referral/commission earnings. It inherits `PausableUpgradeable` in

addition to the standard `AccessControlUpgradeable` and `EIP712Upgradeable` base, allowing claims to be halted instantly in an emergency while admin recovery functions remain available.

Two claim paths are supported: a single-receiver `claimCommission` and a multi-receiver `claimCommissionWithSplit`. The split-claim path binds the entire `receivers/amounts` array into the signed digest via `splitsHash = keccak256(abi.encode(receivers, amounts))`, so a relayer cannot alter the distribution after the backend has signed it. Per-claim, the contract independently re-sums the provided amounts and compares against the signed `totalAmount`, and separately re-checks the vault's live balance before transferring — providing defense-in-depth against any single validation step being bypassed. Split claims additionally reject contract-code receivers outright, narrowing the attack surface for reentrancy or fallback-triggered side effects during distribution.

Nonces are tracked per-agent (`msg.sender`) and incremented before any external call in every verification path, consistent with checks-effects-interactions ordering. Admin deposit and emergency-withdraw functions are likewise signature-gated rather than role-gated alone, matching the security posture of the other OQA contracts.

Contract Structure

OQAFixedDeposits

- **Storage:** EIP-7201 namespaced `OQAFixedDepositsStorage` struct containing asset, custodian, agent contract, APY map, per-user deposit limits, penalty tiers, cycle config, whitelist states, deposit records, and per-user deposit ID arrays — no external auth contract reference
- **Roles:** `DEFAULT_ADMIN_ROLE`, `OPERATOR_ROLE`, `PAYOUT_ROLE`
- **Initialization:** Sets asset, custodian, signer, agent contract, max agent referral rate, penalty tiers, and day-based cycle defaults
- **Core Functions:**
 - `addToWhitelist` / `removeFromWhitelist`: EIP-712 signature-gated whitelist management, keyed directly by address
 - `deposit` / `increaseDeposit`: Whitelist-gated deposit creation and top-up, enforcing per-user min/max limits
 - `consumeWithdraw` (two overloads): Called by payout contract; handles full-principal and partial-amount withdrawal, applies penalty/tax, manages yield checkpointing or clearing based on maturity
 - `consumeYieldClaim` / `compoundYieldClaims`: Called by payout contract; validates claim frequency and claimable ceiling, or bundles multiple deposits' yield into a new compounded deposit
 - `depositOnBehalf`: Signature-gated admin deposit creation with deposit-on-behalf liability tracking
 - `setUserApy`: Signature-gated APY assignment with per-deposit yield checkpointing before rate change
- **Internals:** `_calcYieldSince`, `_updateYield`, `_checkpointPendingYield`, `_unlockPendingYield`, `_clearYieldAfterEarlyWithdraw`, `_latestUnlockedYieldCheckpoint`, `_nextYieldUnlockTime`, `_enforceDepositAmount`, `_enforceRemainingDepositAmount`, `_settleDepositFunds`
- **Getters:** Deposit info, deposit APY, claimable yield, whitelist status, user deposit limits, cycle config, deposit-on-behalf liability

OQAFixedDepositsPayout

- **Storage:** EIP-7201 namespaced `PayoutStorage` struct containing deposits contract reference, signer, and per-address nonces; inherits `AsyncRequest` for request lifecycle state
 - **Roles:** `DEFAULT_ADMIN_ROLE`, `OPERATOR_ROLE`
 - **Initialization:** Sets deposits contract, signer, and admin roles; initialises EIP-712 and `AsyncRequest`
 - **Core Functions:**
 - `requestWithdraw` (two overloads): Full or partial principal withdrawal request, enqueues `PRINCIPAL_WITHDRAWAL` async request
 - `requestClaimYield` / `requestClaimYields`: Single or bulk yield claim requests
 - `compoundYield`: Converts claimable yield + optional wallet top-up into a new deposit, routing top-up directly to custodian
 - `processWithdrawRequests`: EIP-712 signature-gated request fulfilment
 - `completePrincipalWithdrawRequests` / `completeYieldClaimRequests`: Completes fulfilled requests, distributes single-asset payout via basis-point split
 - `bulkWithdrawYieldClaims`: Batch completion of fulfilled yield-claim requests with balance pre-check
 - `adminDepositToken` / `withdrawToken`: Signature-gated admin fund management
 - **Internals:** `_requestWithdraw`, `_distribute`, `_validateRequestTypes`, `_validateFulfilledYieldClaimRequests`
 - **Getters:** Request info, deposits contract address, signer, user nonce, domain separator
-

OQAAgentPayoutVault

- **Storage:** EIP-7201 namespaced `AgentPayoutVaultStorage` struct containing asset token and per-agent nonces
 - **Roles:** `PAYOUT_SIGNER_ROLE`, `ADMIN_ROLE` (aliased to `DEFAULT_ADMIN_ROLE`), `UPGRADER_ROLE`
 - **Initialization:** Sets asset token, admin, and payout signer roles; initialises EIP-712 and `Pausable`
 - **Core Functions:**
 - `claimCommission`: Single-receiver commission claim gated by EIP-712 signature
 - `claimCommissionWithSplit`: Multi-receiver split claim with signature-bound `splitsHash`, per-receiver amount validation, and contract-receiver rejection
 - `deposit`: Signature-gated admin fund top-up
 - `emergencyWithdraw`: Signature-gated admin emergency withdrawal
 - `pause` / `unpause`: Admin-only emergency halt of claim functions
 - `setAsset`: Admin-only asset token update
 - **Internals:** `_verifyClaim`, `_verifyClaimWithSplit`, `_verifyAdminDeposit`, `_verifyAdminWithdraw`
 - **Getters:** `asset`, `nonces`, `DOMAIN_SEPARATOR`
-